

Tehnologii Cloud în Azure

December 7, 2025

Abstract

Demonstrație practică a conceptelor de bază în Cloud (Compute, Orchestrare) și explorarea câtorva tehnologii folosite în industrie (Serverless, CI/CD, IaC, Monitorizare).

Contents

1	Precondiții	3
1.1	Azure pentru studenți	3
1.2	Configurarea mediului de lucru	3
1.2.1	Opțiunea A: Azure Cloud Shell	3
1.2.2	Opțiunea B: Terminal local	3
2	Infrastructura Cloud	4
2.1	Zone și regiuni	4
2.1.1	Identificarea zonelor disponibile pentru subscripție	5
2.2	Grupuri de resurse	5
2.2.1	Crearea unui grup de resurse	5
2.2.2	Ștergerea unui grup de resurse	5
2.3	Mașina virtuală (Virtual Machine - VM)	5
2.3.1	Resurse computaționale (SKU)	6
2.3.2	Imagini ale sistemului de operare	6
2.3.3	Crearea unui VM	6
2.3.4	Verificare și Conectare	7
2.3.5	Ștergerea unui VM	7
2.4	Scalabilitate și Load Balancing (VM Scale Sets)	7
2.4.1	Crearea unui cluster de VM-uri	8
2.4.2	Demonstrația scalării manuale	8
2.4.3	Ștergerea resurselor	9
2.5	Kubernetes (AKS)	9
2.5.1	Arhitectura unui cluster	9
2.5.2	Azure Kubernetes Service	9
2.5.3	Înregistrarea furnizorului de resurse	9
2.5.4	Crearea unui cluster gestionat	10
2.5.5	Conectarea terminalului la cluster	10
2.5.6	Lansarea unei Aplicații Web (AKS Hello World)	10
2.5.7	Comenzi comune în Kubernetes	11
2.5.8	Ștergerea resurselor	12

3	Module optionale	13
3.1	Azure Functions (Serverless)	13
3.1.1	Exemplu practic: Serverless HTTP Endpoint	13
3.1.2	Adăugarea și publicarea codului personalizat	14
3.1.3	Ștergerea resurselor	14
3.2	Terraform (IaC)	14
3.2.1	Task: Deploy Container "Hello World"	15
3.2.2	Aplicare	16
3.2.3	Ștergerea resurselor	16
3.3	Ansible (Configuration Management)	16
3.3.1	Provizionarea infrastructurii	16
3.3.2	Definirea inventarului	17
3.3.3	Automatizare - Web Server & System Hardening	17
3.3.4	Execuție și validare	18
3.3.5	Ștergerea resurselor	18
3.4	Jenkins (CI/CD Clasic)	18
3.4.1	Jenkins în ecosistemul Azure	18
3.4.2	Lansarea Jenkins în Cloud (Azure Container Instances)	19
3.4.3	Accesare și configurare inițială	19
3.4.4	Simulare Pipeline Azure (Groovy)	19
3.4.5	Execuția și monitorizarea pipeline-ului	20
3.4.6	Ștergerea resurselor	21
3.5	GitHub Actions	21
3.5.1	Exemplu de configurare	22
3.6	Stiva ELK & Grafana	22
3.6.1	Configurare	22
3.6.2	Rulare și vizualizare	23
3.6.3	Oprire resurse	24

1 Precondiții

1.1 Azure pentru studenți

Înainte de a începe, studenții pot beneficia în fiecare an de un credit gratuit de 100\$ valabil timp de un an pentru a explora serviciile Azure, fără a fi necesar un card de credit la înregistrare.

Pași necesari pentru activare:

1. Se accesează pagina oficială: <https://azure.microsoft.com/en-us/free/students>.
2. Se apasă butonul verde **"Start free"**.
3. Se realizează autentificarea folosind adresa de email instituțională (oferită de universitate).
4. Se parcurge procesul de verificare a identității.
5. Odată activat contul, abonamentul *"Azure for Students"* va fi vizibil în portal și poate fi folosit pentru a crea resursele din acest laborator.

1.2 Configurarea mediului de lucru

Pentru a executa comenzile din acest laborator, există două opțiuni principale:

1.2.1 Opțiunea A: Azure Cloud Shell

Aceasta este cea mai simplă metodă, deoarece nu necesită instalarea niciunui software local.

1. Se accesează portal.azure.com.
2. Lângă bara de căutare din partea de sus, se apasă pe iconița **Terminal** (arată ca un `>_`).
3. **Avantaj:** Autentificarea este deja realizată (`az login` nu este necesar), iar `kubectl`, `terraform` și `ansible` sunt preinstalate.

La prima accesare, terminalul va cere următoarele:

1. Să se selecteze tipul de terminal. Se va alege **Bash** pentru a simula un terminal similar cu cele folosite până acum la laborator.
2. Să se selecteze dacă se dorește persistență pentru fișierele create în cadrul terminalului și subscripția folosită. Pentru stocare se va selecta **No storage account required**, iar pentru subscripție se va selecta **Azure for students**.

1.2.2 Opțiunea B: Terminal local

Dacă se preferă lucrul de pe propriul laptop:

1. **Instalare Azure CLI:** Se descarcă de pe site-ul oficial Microsoft.
2. **Autentificare:** Se deschide terminalul și se rulează:

```
az login
```

3. **Instalare kubectl:**

```
az aks install-cli
```

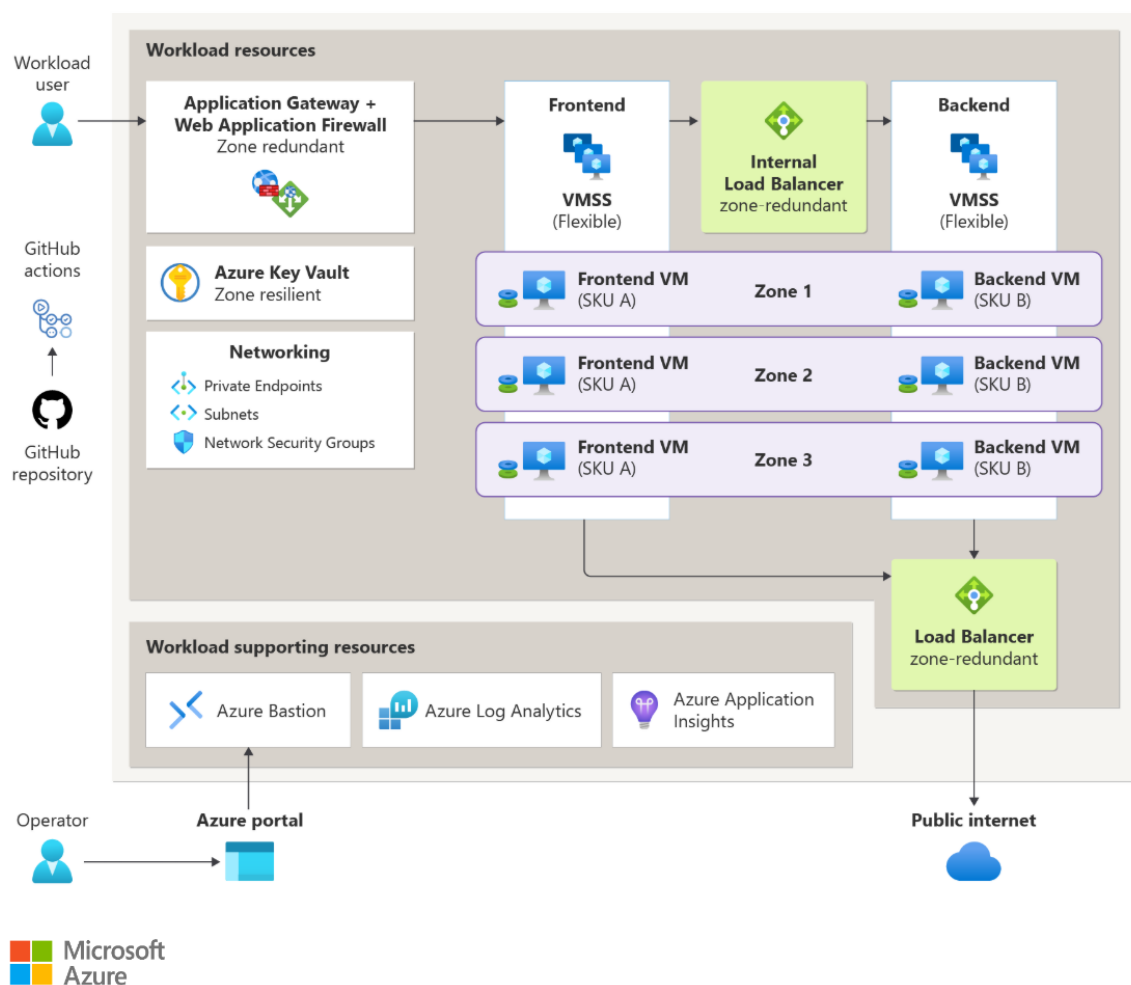


Figure 1: Exemple de tehnologii folosite pentru deployment-ul unei aplicații Web în Azure. Sursă [Microsoft Learn](#).

2 Infrastructura Cloud

Modulele prezentate în continuare constituie fundamentele infrastructurii cloud, fiind recomandată parcurgerea lor în ordinea indicată.

2.1 Zone și regiuni

Regiuni Azure (Regions) O regiune reprezintă o arie geografică delimitată (ex: *West Europe*, *East US*, *Italy North*) care conține unul sau mai multe centre de date interconectate printr-o rețea dedicată cu latență redusă. Alegerea regiunii este critică din două motive:

- **Performanță:** Se recomandă alegerea unei regiuni cât mai apropiate fizic de utilizatorii finali pentru a minimiza latența rețelei.
- **Conformitatea datelor:** Anumite reglementări (ex: GDPR) pot impune ca datele să nu părăsească o anumită jurisdicție geografică.

Zone de Disponibilitate (Availability Zones) În cadrul unei regiuni, Azure oferă **Zone de Disponibilitate**. Acestea sunt locații fizice unice, complet separate, care dispun de alimentare cu energie, răcire și rețelistică independente.

- O regiune poate avea mai multe zone (de obicei 3).
- Scopul lor este de a proteja aplicațiile împotriva defecțiunilor la nivel de centru de date (incendii, inundații, pene de curent majore). Dacă o zonă devine indisponibilă, celelalte continuă să funcționeze.

2.1.1 Identificarea zonelor disponibile pentru subscripție

Întrucât subscripțiile Azure educaționale sau de tip "Free Trial" impun restricții geografice asupra resurselor, nu toate zonele globale Azure sunt accesibile. Prin urmare, este necesară identificarea regiunilor permise pentru crearea resurselor.

Aceste informații se pot consulta în portalul Azure, secțiunea [Allowed resource deployment regions](#), sau direct din terminal.

```
az policy assignment list --query "[].{PolicyName:displayName, Parameters:parameters}" --output json
```

În rezultatul comenzii se poate observa o intrare (parametru) denumită **listOfAllowedLocations**. Aceasta conține lista zonelor disponibile pentru subscripția curentă, determinate inclusiv pe baza proximității geografice față de utilizator.

În exemplele următoare, se va considera **italynorth** ca fiind regiunea selectată pentru desfășurarea experimentelor.

2.2 Grupuri de resurse

2.2.1 Crearea unui grup de resurse

În ecosistemul Azure, orice resursă trebuie să fie inclusă într-un container logic numit *Resource Group*. Acesta facilitează gestionarea ciclului de viață, monitorizarea și aplicarea politicilor de acces pentru resursele aferente.

```
az group create --name CloudLabRG --location italnorth
```

2.2.2 Ștergerea unui grup de resurse

Atunci când resursele nu mai sunt necesare, se recomandă ștergerea grupului. Această acțiune va șterge automat toate resursele conținute în acesta (VM-uri, discuri, IP-uri), prevenind costurile suplimentare.

```
az group delete --name CloudLabRG --yes --no-wait
```

2.3 Mașina virtuală (Virtual Machine - VM)

Deși conceptul fundamental rămâne același — o instanță a unui sistem de operare decuplată de hardware — o mașină virtuală în Azure diferă semnificativ de una locală prin arhitectură și orchestrare. Dacă un VM local este constrâns de resursele stației de lucru gazdă, un VM în Azure beneficiază de:

- **Arhitectură pe Hipervizor de Tip 1:** Rulează direct pe hardware-ul centrelor de date Azure (prin intermediul Azure Hypervisor), eliminând supraîncărcarea (overhead-ul) unui sistem de operare gazdă convențional.

- **Scalabilitate Elastică:** Utilizatorul nu este limitat de hardware-ul propriu; se poate scala vertical (creșterea capacității CPU/RAM a instanței) sau orizontal (autoscale cu sute de instanțe simultane) la cerere.
- **High Availability (Reziliență):** Componenta *Fabric Controller* monitorizează starea hardware-ului și migrează automat VM-ul în caz de avarie fizică, garantând SLA-urile (Service Level Agreements) pentru disponibilitate.
- **Networking Definit prin Software (SDN):** VM-ul este integrat nativ într-o rețea virtuală (VNet), permițând topologii complexe și conectivitate hibridă, depășind limitările unei simple conexiuni la placa de rețea locală.

Exemple de scenarii de utilizare:

- Necesitatea unui control complet asupra configurației sistemului de operare.
- Migrarea aplicațiilor existente ("Lift and shift") care nu sunt containerizate.
- Găzduirea bazelor de date care necesită setări specifice la nivel de sistem de fișiere sau disc.

2.3.1 Resurse computaționale (SKU)

Pentru a stabili resursele hardware alocate (vCPU, RAM, IOPS), se selectează o "dimensiune" (Size/SKU). Lista completă și specificațiile se regăsesc în [documentația oficială Azure](#). Disponibilitatea acestora poate varia în funcție de regiune. Pentru a vizualiza dimensiunile disponibile în regiunea selectată, se utilizează comanda:

```
az vm list-sizes --location italnorth --output table
```

2.3.2 Imagini ale sistemului de operare

Azure oferă o galerie extinsă (Marketplace) de imagini pre-configurate pentru sistemul de operare. Lista imaginilor populare se poate consulta prin:

```
az vm image list
```

2.3.3 Crearea unui VM

În exemplul următor, se va proviziona un server Ubuntu. Procesul va genera automat resursele adiacente necesare: un **VNet** (Rețea Virtuală), o interfață de rețea și un **IP Public**.

```
az vm create \
  --resource-group CloudLabRG \
  --name MyFirstVM \
  --size Standard_B2als_v2 \
  --image Ubuntu2404 \
  --admin-username azureuser \
  --generate-ssh-keys \
  --public-ip-sku Standard \
  --location italnorth
```

Notă: Comanda poate eșua dacă regiunea selectată nu are capacitate suficientă pentru dimensiunea cerută (Size). În acest caz, se recomandă încercarea unei alte regiuni sau a unei alte dimensiuni.

Alternativă prin Portalul Azure (GUI)

1. În bara de căutare, se introduce **"Virtual machines"** și se selectează serviciul.
2. Se acționează butonul **Create** → **Azure virtual machine**.
3. **Tab-ul Basics:**
 - **Resource Group:** Se selectează CloudLabRG (sau se creează unul nou).
 - **Virtual machine name:** MyFirstVM.
 - **Image:** Ubuntu Server 24.04 LTS (sau versiunea preferată).
 - **Size:** Standard_B1s (opțiune economică recomandată pentru teste).
4. Se apasă **Review + create**, urmat de **Create** după validare.

2.3.4 Verificare și Conectare

Pentru a obține adresa IP publică alocată mașinii virtuale:

```
az vm list-ip-addresses --resource-group CloudLabRG --name MyFirstVM --output table
```

Conectarea se realizează prin SSH, utilizând cheia generată automat (stocată implicit în `~/.ssh/`):

```
ssh azureuser@<IP-Public-VM>
```

2.3.5 Ștergerea unui VM

După finalizarea experimentelor, este esențială ștergerea resurselor pentru a evita consumul creditului disponibil.

Atenție: Comanda de mai jos șterge doar obiectul VM, dar păstrează discurile și interfețele de rețea (care continuă să genereze costuri). Pentru o curățare completă, se recomandă ștergerea grupului de resurse (vezi secțiunea 1.2.2). Dacă se dorește ștergerea doar a VM-ului:

```
az vm delete --resource-group CloudLabRG --name MyFirstVM --yes
```

2.4 Scalabilitate și Load Balancing (VM Scale Sets)

În mediile de producție moderne, o singură mașină virtuală reprezintă adesea un punct unic de eșec (Single Point of Failure) și un potențial bottleneck pentru performanță. Pentru a adresa aceste probleme, Azure oferă soluții de scalare orizontală și distribuție a traficului.

Scalabilitatea orizontală vs. verticală

- **Scalarea Verticală (Scale Up):** Presupune creșterea puterii de calcul a unei singure mașini (ex: trecerea de la 2 vCPU la 4 vCPU). Aceasta necesită, de obicei, repornirea mașinii (down-time) și are limite fizice.
- **Scalarea Orizontală (Scale Out):** Presupune adăugarea mai multor mașini identice pentru a prelua sarcina. Aceasta oferă o elasticitate teoretic infinită și nu necesită oprirea serviciului.

Virtual Machine Scale Sets (VMSS) VMSS este un serviciu de calcul care permite implementarea și gestionarea unui grup de mașini virtuale identice (aceeași imagine OS, aceleași configurări). Beneficiile majore includ:

- **Elasticitate:** Numărul de instanțe VM poate crește sau scădea automat ca răspuns la cerere (ex: dacă utilizare CPU este mai mare de 70%, se adaugă 2 instanțe).
- **Administrare centralizată:** Se configurează o singură dată șablonul, iar Azure replică acest șablon pe mii de instanțe.

Azure Load Balancer Acesta este punctul unic de intrare pentru clienți. Load Balancer-ul funcționează la nivelul 4 (Transport Layer - TCP/UDP) și are rolul de a distribui traficul entrant către instanțele VM sănătoase din cadrul Scale Set-ului. Un mecanism critic este **Health Probe**: Load Balancer-ul verifică periodic dacă VM-urile răspund. Dacă o instanță eșuează, traficul este redirectionat automat doar către instanțele funcționale.

2.4.1 Crearea unui cluster de VM-uri

În exemplul următor, se va provisiona un set de scalare care pornește inițial cu 2 instanțe. Azure va crea automat și un Load Balancer pentru a distribui traficul între acestea pe portul 80.

```
az vmss create \
  --resource-group CloudLabRG \
  --name LabScaleSet \
  --image Ubuntu2404 \
  --vm-sku Standard_B2als_v2 \
  --upgrade-policy-mode automatic \
  --admin-username azureuser \
  --generate-ssh-keys \
  --instance-count 2 \
  --backend-port 80 \
  --location italnorth
```

Explicarea parametrilor:

- `--instance-count 2`: Definește capacitatea inițială a clusterului.
- `--backend-port 80`: Instruiește Load Balancer-ul să trimită traficul HTTP către portul 80 al fiecărui VM.
- `--upgrade-policy-mode automatic`: Asigură aplicarea automată a actualizărilor asupra instanțelor, fără intervenție manuală.

Alternativă prin Portalul Azure (GUI)

1. Se caută serviciul "**Virtual machine scale sets**".
2. Se apasă **Create**.
3. La secțiunea **Orchestration**, se selectează **Flexible** (pentru o disponibilitate ridicată și control granular).
4. La tab-ul **Networking**, este esențială bifarea opțiunii **Use a load balancer** pentru a permite distribuirea traficului.

2.4.2 Demonstrația scalării manuale

Deși VMSS suportă *autoscaling* (bazat pe metrici), pentru acest exercițiu se va simula o creștere a cererii prin scalare manuală. Se va modifica capacitatea clusterului de la 2 la 4 instanțe.

```
# Scalare manuala (Scale Out)
az vmss scale --resource-group CloudLabRG --name LabScaleSet --new-capacity 4

# Verificarea noilor instanțe create
az vmss list-instances --resource-group CloudLabRG --name LabScaleSet --output table
```

În urma rulării comenzii de listare, se va observa apariția a două noi instanțe care sunt în proces de *Creating* sau *Running*. Load Balancer-ul va începe automat să trimită trafic către acestea imediat ce devin operaționale.

2.4.3 Ștergerea resurselor

Deoarece un Scale Set cu 4 instanțe consumă resurse semnificative, se recomandă ștergerea acestuia imediat după finalizarea testelor.

```
az vmss delete --resource-group CloudLabRG --name LabScaleSet
```

2.5 Kubernetes (AKS)

În evoluția infrastructurii, dacă o Mașină Virtuală (VM) abstractizează hardware-ul pentru a rula un sistem de operare complet, containerele abstractizează sistemul de operare pentru a rula aplicații izolate și portabile.

Kubernetes (K8s) reprezintă standardul industrial pentru **orchestrarea containerelor**. Acesta funcționează ca un sistem de operare pentru cluster, gestionând ciclul de viață al aplicațiilor containerizate. Într-un mediu de producție, Kubernetes automatizează operațiuni critice:

- **Desired State Configuration (Reconciliere):** Utilizatorul definește starea finală dorită (ex: „5 replici ale aplicației X”). Kubernetes monitorizează continuu sistemul și, în cazul unei discrepante (ex: un container se oprește forțat), inițiază automat procesul de *Self-healing* pentru a restabili starea declarată.
- **Scheduling Intelligent:** Sistemul decide automat distribuția containerelor pe nodurile de calcul (servere) în funcție de resursele disponibile (CPU/RAM) și de constrângerile definite.
- **Service Discovery & Load Balancing:** Traficul de rețea este distribuit dinamic către containere, oferind o adresă stabilă (DNS) independent de adresa IP volatilă a containerelor individuale.

2.5.1 Arhitectura unui cluster

Componentele principale sunt împărțite în două planuri logice:

1. **Pod:** Cea mai mică unitate de execuție în Kubernetes. Un Pod încapsulează unul sau mai multe containere care împart stocarea și rețeaua.
2. **Control Plane (Creierul):** Responsabil pentru deciziile globale (planificare, detectare și răspuns la evenimente). Conține API Server, etcd (baza de date) și controllere.
3. **Worker Nodes (Executanții):** Mașinile (VM-uri sau fizice) unde rulează efectiv containerele/pod-urile.

2.5.2 Azure Kubernetes Service

Instalarea manuală a Kubernetes (*Self-Hosted*) implică o complexitate operațională ridicată pentru menținerea Control Plane-ului. **AKS** este o soluție de tip *Managed Kubernetes*. În acest model, Microsoft Azure gestionează gratuit **Control Plane-ul** (actualizări, disponibilitate), iar utilizatorul gestionează și plătește doar pentru **Worker Nodes**.

2.5.3 Înregistrarea furnizorului de resurse

Înainte de a proviziona clusterul, este necesară o etapă preliminară de configurare a subscripției. Azure funcționează pe baza unor **Resource Providers** (Furnizori de Resurse). O subscripție nouă nu are active implicit toate serviciile posibile, pentru a reduce ”zgomotul” și a menține securitatea (”Principiul minimului necesar”).

Serviciul AKS aparține furnizorului `Microsoft.ContainerService`. Rularea comenzii de mai jos este echivalentă cu instalarea unui "driver" sau activarea unui "plugin", semnalând platformei Azure că această subscripție este autorizată să solicite și să gestioneze resurse Kubernetes.

```
az provider register --namespace Microsoft.ContainerService
```

Se poate verifica finalizarea înregistrării rulând:

```
az provider show -n Microsoft.ContainerService --query "registrationState"
```

Se așteaptă starea "Registered".

2.5.4 Crearea unui cluster gestionat

Următoarea comandă provizionează un cluster simplu, cu un singur nod, optimizat pentru experimentat. *Notă: Procesul de creare poate dura între 5 și 10 minute.*

```
az aks create \
  --resource-group CloudLabRG \
  --name LabAKSCluster \
  --node-count 1 \
  --generate-ssh-keys \
  --tier free
```

Alternativă prin Portalul Azure (GUI) Se caută serviciul "**Kubernetes services**". La creare, se selectează configurația presetată **Dev/Test** (pentru a reduce costurile) și se setează manual *Node count* la valoarea 1.

2.5.5 Conectarea terminalului la cluster

Pentru a interacționa cu clusterul, este necesară configurarea utilitarului `kubectl` cu datele de acces ale noului cluster:

```
az aks get-credentials --resource-group CloudLabRG --name LabAKSCluster
```

2.5.6 Lansarea unei Aplicații Web (AKS Hello World)

AKS Hello World este o aplicație web simplă, menținută de Microsoft, care afișează un mesaj de bun venit.

Pentru a defini infrastructura se va folosi un fișier manifest **YAML** care conține:

- **Deployment:** Definește aplicația și versiunea imaginii.
- **Service:** Expune aplicația către internet printr-un Load Balancer.

Pasul 1: Crearea fișierului manifest Se rulează comanda de mai jos pentru a genera fișierul `aks-app.yaml`.

```
cat <<EOF > aks-app.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: aks-helloworld
spec:
  replicas: 1
  selector:
    matchLabels:
      app: aks-helloworld
```

```

template:
  metadata:
    labels:
      app: aks-helloworld
  spec:
    nodeSelector:
      "kubernetes.io/os": linux
    containers:
      - name: aks-helloworld
        # Imagine oficiala Microsoft stabila pentru testare
        image: mcr.microsoft.com/azuredocs/aks-helloworld:v1
        ports:
          - containerPort: 80
        env:
          - name: TITLE
            value: "Salutare din Laboratorul Azure!"
---
apiVersion: v1
kind: Service
metadata:
  name: aks-helloworld
spec:
  type: LoadBalancer
  ports:
    - port: 80
  selector:
    app: aks-helloworld
EOF

```

Pasul 2: Aplicarea configurației Se aplică configurarea pe clusterul Kubernetes:

```
kubectl apply -f aks-app.yaml
```

Pasul 3: Monitorizarea și accesarea Alocarea unui IP Public de către Azure Load Balancer necesită timp. Se poate monitoriza starea serviciului folosind parametrul `--watch`:

```
kubectl get service azure-vote-front --watch
```

Se așteaptă până când coloana `EXTERNAL-IP` se modifică din `<pending>` într-o adresă IP numerică. Pentru a opri monitorizarea, se apasă `Ctrl + C`. Adresa IP obținută se introduce în browser pentru a accesa aplicația.

2.5.7 Comenzi comune în Kubernetes

1. Verificarea Nodurilor (Infrastructura) Primul pas este confirmarea faptului că nodurile de calcul sunt operaționale și pregătite să primească sarcini (starea *Ready*).

```
kubectl get nodes
```

2. Verificarea pod-urilor (Aplicațiile) Se afișează lista tuturor pod-urilor din namespace-ul curent.

```
kubectl get pods
```

Se urmărește coloana **STATUS**:

- **Running**: Containerul funcționează normal.
- **Pending**: Pod-ul așteaptă alocarea resurselor (scheduling).

- **ContainerCreating**: Se descarcă imaginea containerului.
- **CrashLoopBackOff** sau **Error**: Aplicația a eșuat și Kubernetes încearcă să o repornească.

Pentru a vedea pe ce nod rulează fiecare pod și care este IP-ul său intern, se adaugă parametrul `-o wide`:

```
kubectl get pods -o wide
```

3. Verificarea serviciilor (Rețeaua) Se listează punctele de acces la aplicații. Aici se poate distinge între serviciile interne și cele expuse public.

```
kubectl get services
```

În lista returnată, tipul *ClusterIP* indică un serviciu accesibil doar din interiorul clusterului (ex: *Redis*), în timp ce *LoadBalancer* indică expunerea publică.

4. Depanare și Detalii Avansate În situația în care un pod nu pornește sau funcționează defectuos, există două comenzi critice pentru investigație:

A. Inspectarea detaliată a configurației și a evenimentelor: Comanda `describe` oferă o radiografie completă a obiectului, inclusiv istoricul evenimentelor (ex: de ce nu s-a putut descărca imaginea sau de ce nu a trecut testul de sănătate).

```
# <nume-pod> se ia din rezultatul comenzii 'kubectl get pods'
kubectl describe pod <nume-pod>
```

B. Vizualizarea log-urilor aplicației: Se afișează ieșirea standard (stdout/stderr) a containerului, similar cu vizualizarea terminalului unei aplicații locale.

```
kubectl logs <nume-pod>
```

2.5.8 Ștergerea resurselor

Este critică ștergerea grupului de resurse la final, deoarece clusterul AKS și Load Balancer-ul asociat generează costuri chiar și în repaus.

```
az group delete --name CloudLabRG --yes --no-wait
```

3 Module optionale

3.1 Azure Functions (Serverless)

Serverless Computing reprezintă o paradigmă de execuție bazată pe evenimente (*event-driven*). Aceasta nu implică dispariția serverelor, ci abstractizarea lor totală. Se dezvoltă secvențe de cod ("funcții") care răspund la evenimente specifice, iar platforma Azure gestionează automat alocarea resurselor, actualizările de securitate și scalarea.

Concepte Cheie:

- **Function App:** Unitatea logică de organizare. Găzduiește una sau mai multe funcții care partajează configurații (ex: variabile de mediu) și resurse de calcul.
- **Triggers (Declanșatori):** Evenimentul care inițiază execuția codului (ex: o cerere HTTP, un fișier încărcat în Blob Storage, un mesaj în coadă).
- **Consumption Plan:** Modelul de "Scale-to-Zero". În absența apelării funcțiilor, nu există instanțe active, iar costul este nul. Facturarea se realizează la nivel de milisecundă (GB-secunde).
- **Bindings (Legături):** Modul declarativ prin care se conectează datele de intrare/ieșire, eliminând necesitatea codului redundant de infrastructură.

Utilizare și beneficii:

- **Arhitecturi reactive:** Procesarea în timp real a datelor (ex: redimensionarea unei imagini imediat după încărcare).
- **Microservicii eficiente:** API-uri ușoare care rulează independent.
- **Scalabilitate extremă:** Gestionarea automată a traficului, de la zero la zeci de mii de cereri concurente.

3.1.1 Exemplu practic: Serverless HTTP Endpoint

În exemplul următor, se utilizează variabile pentru a asigura unicitatea numelor și consistența între comenzi, evitând astfel erorile de configurare.

```
# 0. Definire Variabile (pentru consistenta)
RND=$RANDOM
GRP="ServerlessRG-$RND"
LOC="italynorth"
STO="funcstore$RND"
APP="MyLabFuncApp-$RND"

echo "Se creeaza resursele in grupul: $GRP..."

# 1. Creare Resource Group
az group create --name $GRP --location $LOC

# 2. Creare Storage Account (Obligativu pentru Functions)
az storage account create \
  --name $STO \
  --location $LOC \
  --resource-group $GRP \
  --sku Standard_LRS

# 3. Se inregistreaza providerul Microsoft.Web
az provider register --namespace Microsoft.Web
```

```
# 3. Creare Function App (Consumption Plan)
# Se utilizeaza runtime-ul Node.js v24
az functionapp create \
  --resource-group $GRP \
  --consumption-plan-location $LOC \
  --runtime node \
  --runtime-version 24 \
  --functions-version 4 \
  --name $APP \
  --storage-account $STO

# 4. Verificare (Smoke Test)
FUNC_URL="https://$APP.azurewebsites.net"
echo "Aplicatia este activa la: $FUNC_URL"

# Se testeaza raspunsul serverului
curl -I $FUNC_URL
```

3.1.2 Adăugarea și publicarea codului personalizat

După crearea infrastructurii, aplicația Function App este un container gol. Pentru a executa logică specifică, se generează un proiect local, se adaugă o funcție și se publică în cloud folosind Azure Functions Core Tools ('func').

```
# 1. Initializare proiect local
# Se creeaza un director dedicat si se initializeaza structura Node.js
mkdir MyLocalFunc
cd MyLocalFunc
func init --worker-runtime node --language javascript

# 2. Generare Functie (HTTP Trigger)
# Se creeaza o functie care raspunde la cereri HTTP anonime
func new --name MyCustomTrigger --template "HTTP trigger" --authlevel anonymous

# 3. Publicare in Azure
# Se incarca codul local in aplicatia creata anterior (variabila $APP)
func azure functionapp publish $APP

# 4. Validare Functionalitate
# Se apeleaza endpoint-ul specific functiei nou create
# URL-ul va fi de forma: https://<NumeAplicatie>.azurewebsites.net/api/MyCustomTrigger
curl "https://$APP.azurewebsites.net/api/MyCustomTrigger?name=CloudLabUser"
```

3.1.3 Ștergerea resurselor

```
az group delete --name $GRP --yes --no-wait
```

3.2 Terraform (IaC)

Infrastructure as Code (IaC) Este practica de a gestiona și a proviziona infrastructura prin fișiere de definiție lizibile de către mașină, în locul configurării manuale sau scripturilor interactive.

Concepte cheie:

- **Terraform:** Un instrument open-source creat de HashiCorp, care permite definirea infrastructurii folosind un limbaj declarativ (HCL - HashiCorp Configuration Language).

- **Provider:** Un plugin care permite Terraform să interacționeze cu un API specific (ex: **azurerm** pentru Azure, **aws** pentru Amazon).
- **State File (terraform.tfstate):** O bază de date locală (sau la distanță) care mapează resursele din lumea reală la configurația ta. Permite Terraform să știe ce a creat deja.
- **Declarativ vs Imperativ:** Îi spui lui Terraform *"Vreau 5 servere"*, nu *"Creează un server, apoi încă unul..."*.

De ce se utilizează?

- **Reproductibilitate:** Se poate crea un mediu identic (Dev, Test, Prod) rulând același cod.
- **Controlul Versiunilor:** Fișierele de infrastructură pot fi stocate în Git, permițând auditarea modificărilor și revenirea la versiuni anterioare (Rollback).
- **Cloud-Agnostic:** Permite învățarea unui singur flux de lucru (Workflow) pentru a gestiona resurse pe Azure, AWS, Google Cloud sau VMWare.

3.2.1 Task: Deploy Container "Hello World"

Se creează fișierul `main.tf`:

```
terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 3.0" }
    random = { source = "hashicorp/random", version = "~> 3.0" }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "tf_rg" {
  name     = "TerraformLabRG"
  location = "italynorth"
}

resource "random_string" "dns" {
  length     = 6
  special    = false
  upper      = false
  numeric    = true
}

resource "azurerm_container_group" "app" {
  name                = "tf-app"
  location             = azurerm_resource_group.tf_rg.location
  resource_group_name = azurerm_resource_group.tf_rg.name
  os_type              = "Linux"
  dns_name_label       = "myapp-${random_string.dns.result}"

  container {
    name     = "hello-world"
    image    = "mcr.microsoft.com/azuredocs/aci-helloworld"
    cpu      = "0.5"
    memory   = "1.5"
    ports {
```

```

        port      = 80
        protocol = "TCP"
    }
}
}

output "url" { value = "http://${azurerm_container_group.app.fqdn}" }

```

3.2.2 Aplicare

```

terraform init
terraform apply -auto-approve

```

Pentru a testa ca deployment-ul s-a făcut cu succes se accesează URL-ul obținut ca output.

3.2.3 Ștergerea resurselor

```

terraform destroy -auto-approve

```

3.3 Ansible (Configuration Management)

Ansible este un instrument de **Configuration Management** și automatizare IT, utilizat pentru a aduce serverele într-o stare dorită.

Caracteristici principale:

- **Agentless:** Nu necesită instalarea niciunui software (agent) pe serverele țintă; comunicarea se realizează exclusiv prin protocolul SSH.
- **Idempotență:** Proprietatea de a rula același script de nenumărate ori fără a produce efecte secundare nedorite. Dacă serverul este deja configurat corect, Ansible nu va efectua nicio modificare.
- **Playbook:** Fișierul YAML (inima automatizării) unde sunt descrise sarcinile și stările dorite ale sistemului.
- **Inventory:** Un fișier text care conține lista serverelor (adrese IP sau domenii) pe care Ansible le administrează.

Notă comparativă: Deși Terraform și Ansible pot părea că îndeplinesc funcții similare, ele sunt concepute pentru scopuri diferite. În practică, acestea sunt adesea folosite împreună într-un flux de lucru complementar: Terraform creează infrastructura (provizionare), iar Ansible instalează și configurează aplicațiile pe acea infrastructură (configurare).

3.3.1 Provizionarea infrastructurii

Pentru a avea o țintă reală pentru configurare, se va crea un Mașină Virtuală în Azure. Se deschid porturile 22 (pentru conexiunea Ansible) și 80 (pentru serverul web ce urmează a fi instalat).

```

# 1. Creare Grup de Resurse
az group create --name CloudLabRG --location italnorth

# 2. Creare VM Ubuntu
az vm create \
  --resource-group CloudLabRG \

```



```
--name AnsibleTargetVM \
--image Ubuntu2404 \
--size Standard_B2als_v2 \
--location italnorth \
--admin-username azureuser \
--generate-ssh-keys \
--public-ip-sku Standard

# 3. Se deschide si portul 80 pentru a expune aplicatia de test
az vm open-port --resource-group CloudLabRG --name AnsibleTargetVM --port 80 --priority 1010
```

După finalizarea comenzii de creare a VM-ului, se notează **publicIpAddress** din rezultatul JSON.

3.3.2 Definirea inventarului

Ansible trebuie să știe unde să se conecteze. Se creează un fișier numit `inventory.ini` în care se definește grupul de servere și parametrii de conexiune.

```
[webservers]
<IP-PUBLIC-VM> ansible_user=azureuser ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Înlocuiți `<IP-PUBLIC-VM>` cu adresa IP reală obținută anterior.

3.3.3 Automatizare - Web Server & System Hardening

Se va crea un Playbook (`setup_server.yml`) care transformă un server "gol" într-un server web securizat. **Atenție:** Ordinea operațiunilor este critică. Înainte de a activa firewall-ul (UFW), trebuie permis traficul SSH, altfel conexiunea se va pierde instantaneu.

```
- name: Hardening & Web Server Setup
  hosts: webservers
  become: true # Ruleaza comenzile cu sudo
  vars:
    app_user: "webapp_user"
    http_port: 80

  tasks:
    # --- 1. System Hardening ---
    - name: Create Application User
      user:
        name: "{{ app_user }}"
        state: present
        shell: /bin/bash

    - name: Allow SSH traffic (CRITICAL before enabling UFW)
      ufw:
        rule: allow
        name: OpenSSH

    - name: Allow HTTP traffic
      ufw:
        rule: allow
        port: "{{ http_port }}"
        proto: tcp

    - name: Enable Firewall (UFW)
      ufw:
        state: enabled
        policy: deny
```

```
# --- 2. Application Configuration ---
- name: Install Nginx
  apt:
    name: nginx
    state: present
    update_cache: yes

- name: Start Nginx Service
  service:
    name: nginx
    state: started
    enabled: yes
```

3.3.4 Execuție și validare

Se rulează playbook-ul specificând fișierul de inventar creat.

```
# Rulare Playbook
# Optiunea StrictHostKeyChecking=no evita promptul yes/no la prima conectare
ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i inventory.ini setup_server.yml
```

Dacă totul a funcționat corect, accesarea IP-ului public în browser va afișa pagina implicită Nginx ("Welcome to nginx!").

3.3.5 Ștergerea resurselor

```
az group delete --name CloudLabRG --yes --no-wait
```

3.4 Jenkins (CI/CD Clasic)

CI/CD (Continuous Integration / Continuous Deployment) sunt un set de practici care permit echipelor de dezvoltare să livreze modificări de cod mai frecvent și mai fiabil.

Jenkins: Un server de automatizare open-source. Funcționează ca un orchestrator care execută o serie de pași (Pipeline) ori de câte ori cineva modifică codul sursă.

- **Pipeline:** O definiție a procesului de livrare (Build → Test → Deploy) scrisă în cod (Pipeline as Code).

De ce se utilizează?

- **Eliminarea erorii umane:** Procesele manuale de deployment sunt predispuse la greșeli. Automatizarea asigură consistența.
- **Viteză:** Lansarea unei noi versiuni durează minute, nu ore.
- **Istoric și Audit:** Se poate vedea exact cine a făcut deployment, când și ce versiune.

3.4.1 Jenkins în ecosistemul Azure

Pentru ca Jenkins să poată manipula resurse Azure (să creeze VM-uri sau să facă deploy în Kubernetes), acesta are nevoie de:

1. **Plugins:** Extensii specifice (ex: *Azure CLI Plugin*, *Kubernetes Plugin*).
2. **Service Principal:** O identitate de securitate (un "user robot") care îi oferă drepturi de acces în subscripția Azure.

3.4.2 Lansarea Jenkins în Cloud (Azure Container Instances)

În loc să instalăm Jenkins pe un VM sau local, vom folosi **Azure Container Instances (ACI)**. Acesta este un serviciu "Serverless Containers" ideal pentru a ridica rapid utilitare izolate.

```
# 1. Creare Grup de Resurse (daca nu exista deja)
az group create --name CloudLabRG --location italnorth

# 2. Lansarea instantei Jenkins
# --dns-name-label: Permite accesarea prin URL public
# --ports 8080: Portul standard Jenkins
az container create \
  --resource-group CloudLabRG \
  --name lab-jenkins \
  --image jenkins/jenkins:lts \
  --ports 8080 \
  --dns-name-label jenkins-lab-${RANDOM} \
  --cpu 1 --memory 1.5 \
  --os-type Linux
```

3.4.3 Accesare și configurare inițială

După finalizarea comenzii, Jenkins va fi accesibil public.

1. Se obține URL-ul public (FQDN):

```
az container show --resource-group CloudLabRG --name lab-jenkins --query ipAddress.fqdn --output tsv
```

Se accesează adresa returnată (ex: `jenkins-lab-1234.italynorth...:8080`) în browser.

2. Se obține parola de administrator (care se află în log-urile containerului):

```
az container logs --resource-group CloudLabRG --name lab-jenkins
```

Se caută șirul lung de caractere afișat după "Please use the following password to proceed to installation".

3.4.4 Simulare Pipeline Azure (Groovy)

După deblocarea Jenkins și instalarea plugin-urilor sugerate, se creează un "New Item" de tip **Pipeline**. Scriptul următor simulează un flux complet de livrare a aplicației discutate în capitolul Kubernetes:

```
pipeline {
  agent any
  environment {
    // Definim variabilele specifice aplicatiei utilizate anterior
    IMAGE_NAME = "mcr.microsoft.com/azuredocs/aks-helloworld:v1"
    APP_NAME = "aks-helloworld"
    MANIFEST_FILE = "aks-app.yaml"
  }
  stages {
    stage('1. Code Checkout') {
      steps {
        echo "Se descarca codul sursa pentru aplicatia ${APP_NAME}"
      }
    }
  }
}
```

```

        // Simulare: git clone https://github.com/Azure/aks-
helloworld.git
        echo "Repository clonat cu succes."
    }
}

stage('2. Build Container') {
    steps {
        echo "Se construiește imaginea Docker..."
        // Comanda reala ar fi: docker build -t ${IMAGE_NAME} .
        echo "Building image: ${IMAGE_NAME}"
        echo "Pushing image to Azure Container Registry..."
    }
}

stage('3. Deploy to Dev') {
    steps {
        echo "Se aplica manifestul in clusterul de dezvoltare..."
        // Comanda reala: kubectl apply -f ${MANIFEST_FILE} --
namespace development
        echo "Executing: kubectl apply -f ${MANIFEST_FILE}"
        echo "Status: Serviciul este activ pe un IP intern."
    }
}

stage('4. Manual Approval') {
    steps {
        // Acest pas opreste executia pana la interventia umana
        input message: 'Testele automate au trecut. Lansam in
Productie?', ok: 'DA, Lanseaza!'
    }
}

stage('5. Deploy to Production') {
    steps {
        echo "Se provizioneaza Load Balancer-ul public..."
        // Comanda reala: kubectl apply -f ${MANIFEST_FILE} --
namespace production
        echo "Deployment finalizat."
        echo "Aplicatia ${APP_NAME} este accesibila public!"
    }
}
}
}

```

3.4.5 Execuția și monitorizarea pipeline-ului

Odată ce scriptul a fost introdus în secțiunea de configurare a job-ului, procesul de lansare și validare decurge astfel:

1. **Salvarea configurației:** Se apasă butonul **Save** din josul paginii pentru a persista definiția pipeline-ului.
2. **Lansarea execuției:** În meniul din partea stângă, se apasă butonul **Build Now**. În secțiunea *Build History* (stânga jos), va apărea o nouă intrare (ex: #1) cu o bară de progres intermitentă.
3. **Vizualizarea grafică (Stage View):** În partea centrală a ecranului va apărea reprezentarea

vizuală a pipeline-ului. Fiecare etapă definită în cod (*Build, Deploy Staging, etc.*) va fi reprezentată printr-un cerc colorat.

- **Albastru/Verde:** Execuție în curs sau reușită.
- **Roșu:** Eroare (se pot consulta log-urile pentru detalii).

4. **Interacțiunea cu pasul de aprobare (Manual Approval):** Deoarece scriptul conține instrucțiunea `input`, execuția se va opri (pauză) la stadiul **Manual Approval**. Pătratul aferent va fi galben sau hașurat.

- Se duce cursorul mouse-ului peste etapa "Manual Approval".
- Va apărea un buton sau o fereastră pop-up cu mesajul definit: *"Testele automate au trecut. Lansam in Productie?"*.
- Se apasă butonul **DA, Lanseaza!** pentru a permite continuarea fluxului către producție.

5. **Verificarea log-urilor (Console Output):** Pentru a vedea detaliile tehnice simulate (comenzile `echo`), se dă click pe numărul build-ului (ex: **#1**) din stânga, apoi pe **Console Output**. Aici se va observa istoricul complet al acțiunilor executate de Jenkins.

3.4.6 Ștergerea resurselor

Deoarece ACI taxează la secundă, este importantă ștergerea containerului la final.

```
az container delete --resource-group CloudLabRG --name lab-jenkins --yes
az group delete --name CloudLabRG --yes --no-wait
```

3.5 GitHub Actions

Cloud-Native CI/CD este o abordare modernă unde pipeline-ul de integrare continuă este integrat direct în platforma de versionare a codului (GitHub), eliminând necesitatea unui server de CI separat (precum Jenkins).

Componente cheie:

- **GitHub Actions:** Platforma de automatizare a fluxurilor de lucru oferită de GitHub.
- **Workflow:** Un proces configurabil definit într-un fișier YAML, declanșat de evenimente (ex: push, pull request).
- **Runner:** Serverul care execută job-urile din workflow. Poate fi găzduit de GitHub (ex: `ubuntu-latest`) sau self-hosted.
- **Service Principal (Identitate):** O identitate de securitate în Azure Active Directory utilizată de instrumente automate (precum GitHub) pentru a accesa resursele Azure în mod sigur, fără a folosi credențialele unui utilizator uman.

De ce se utilizează?

- **Proximitatea față de cod:** Configurația pipeline-ului trăiește în același repository cu codul sursă (`.github/workflows`).
- **Mentenanță zero:** Nu trebuie instalat, actualizat sau securizat un server Jenkins.
- **Integrare nativă:** Declanșarea automată a testelor la fiecare Pull Request pentru a preveni introducerea de bug-uri în ramura principală (Main branch).

3.5.1 Exemplu de configurare

Pentru un exemplu de configurare se pot urmări pașii din documentația Microsoft [Deploy to Azure App Service by using GitHub Actions](#).

3.6 Stiva ELK & Grafana

Observability & Centralized Logging Într-un mediu distribuit (microservicii, cloud), este imposibil să te conectezi manual pe fiecare server pentru a citi fișierele de log. Jurnalul trebuie agregat într-un singur loc.

Componente Cheie:

- **Elasticsearch:** Un motor de căutare și analiză distribuit. Este "baza de date" unde sunt stocate logurile. Este optimizat pentru căutări full-text extrem de rapide.
- **Logstash:** Un pipeline de procesare a datelor pe partea de server. Acesta ingerează date din surse multiple simultan, le transformă (parsează) și le trimite către Elasticsearch.
- **Kibana:** O interfață de vizualizare pentru datele din Elasticsearch. Permite interogarea logurilor.
- **Grafana:** O platformă de analiză și monitorizare multi-sursă. Este utilizată pentru a crea dashboard-uri vizuale (grafice, alerte) bazate pe metrici de performanță (CPU, memorie, număr de erori).

De ce se utilizează?

- **Depanare Rapidă:** Când o tranzacție eșuează, se poate căuta ID-ul tranzacției în Kibana pentru a vedea traseul complet prin toate microserviciile.
- **Analiză de Business:** Transformarea logurilor tehnice în date de business (ex: Grafana poate afișa câți bani a generat magazinul în ultima oră bazându-se pe logurile de plată).
- **Alerte:** Notificarea automată a echipei dacă rata de erori depășește un prag critic.

3.6.1 Configurare

Se creează `logstash.conf` pentru a accepta JSON via HTTP:

```
input {
  http {
    port => 5000
    codec => json
  }
}

filter {
  # Daca nu exista un timestamp, adaugam unul bazat pe timpul curent
  if ![timestamp] {
    mutate {
      add_field => { "timestamp" => "%{@timestamp}" }
    }
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "ecommerce-logs-%{+YYYY.MM.dd}"
  }
}
```

```

}
# Afiseaza logurile si in consola pentru debugging
stdout { codec => rubydebug }
}

```

Se creează `docker-compose.yml` (include un generator de trafic automat):

```

version: '3.8'

services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:7.9.2
    environment:
      - discovery.type=single-node
    ports:
      - "9200:9200"

  logstash:
    image: docker.elastic.co/logstash/logstash:7.9.2
    volumes:
      - ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf
    ports:
      - "5000:5000"
    depends_on:
      - elasticsearch

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    depends_on:
      - elasticsearch

  traffic-generator:
    image: alpine
    depends_on:
      - logstash
    command: >
      sh -c '
        apk add --no-cache curl &&
        while true; do
          AMOUNT=$((RANDOM % 500));
          USER_ID=$((RANDOM % 100));
          curl -X POST -H "Content-Type: application/json" \
            -d "{\"user\": \"user_${USER_ID}\", \"amount\": ${AMOUNT}}\" \
            http://logstash:5000;
          sleep 1;
        done
      '

```

3.6.2 Rulare și vizualizare

```
docker-compose up -d
```

Se accesează Grafana la <http://localhost:3000> (admin/admin) și se configurează sursa de date Elasticsearch cu adresa containerului.

3.6.3 Opreire resurse

```
docker-compose down
```